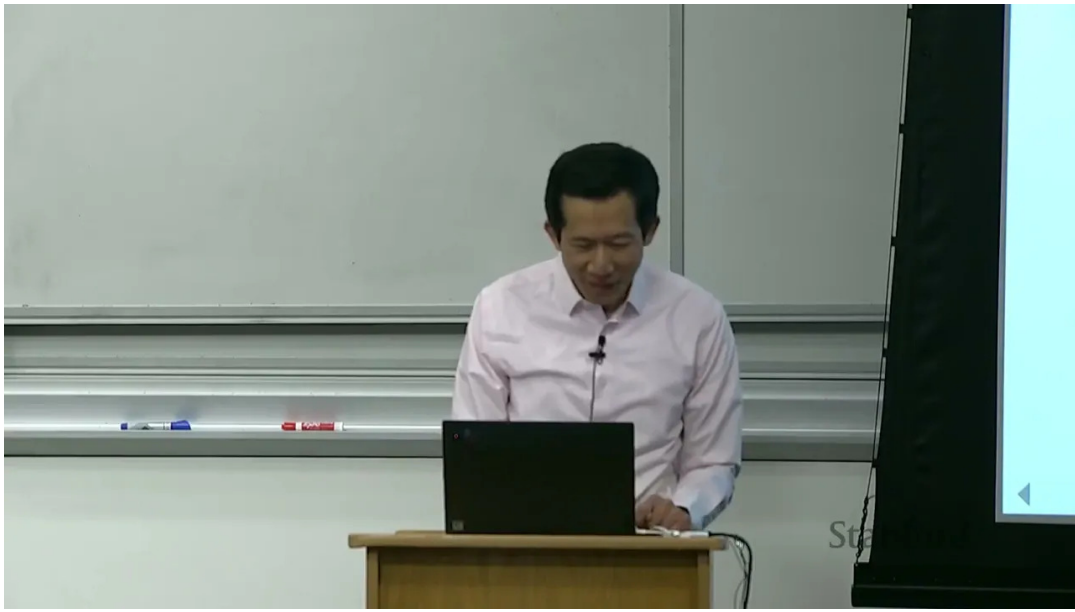


课程讲义

分布式训练代码：通信原语、拓扑与并行实现

Stanford CS336 Spring 2025 · Lecture 8 · Parallelism 2



频道：Stanford Online

发布日期：2025 年 5 月 12 日

视频时长：1 小时 15 分 10 秒

视频链接：<https://www.youtube.com/watch?v=LHpr5ytssLo>

整理说明：依据完整字幕、原始 notebook 画面与代码输出重构。重点是把上一讲的并行公式落实为 `torch.distributed` 程序，并解释同步、带宽测量、状态所有权与拓扑映射。

整理日期：2026 年 7 月 29 日

目录

1	从概念并行到可运行程序	3
1.1	计算层级与通信层级	3
1.2	本章小结	5
2	集合通信的语义	5
2.1	一对多、多对一与全体交换	5
2.2	本章小结	9
3	拓扑、NCCL 与 <code>torch.distributed</code>	10
3.1	同节点快, 跨节点慢	10
3.2	本章小结	12
4	编写并验证 <code>collective</code> 实验	13
4.1	让每个 rank 的输入可辨认	13
4.2	本章小结	14
5	通信基准与带宽口径	15
5.1	异步 CUDA 与同步测量	15
5.2	本章小结	17
6	跨节点通信与分组	18
6.1	高频原语应留在高速域	18
6.2	本章小结	19
7	手写数据并行与状态所有权	19
7.1	复制参数、切数据、归约梯度	19
7.2	本章小结	21
8	张量并行与流水线并行的代码形状	22
8.1	张量并行: 切权重宽度	22
8.2	流水线并行: 切模型深度	24
8.3	本章小结	25
9	把实验代码提升为训练系统	26
9.1	并行、显存与通信要共同回归	26
9.2	本章小结	27

10 分布式程序的调试与验收	27
10.1 启动阶段先证明身份	27
10.2 死锁的系统化排查	28
10.3 数值等价的分层测试	28
10.4 通信性能的对照实验	28
10.5 检查点与弹性	29
10.6 从课堂原型到框架	29
10.7 本章小结	29

1 从概念并行到可运行程序

1.1 计算层级与通信层级

一台分布式训练机器同时有多个层级：线程在 SM 内协作，多个 SM 共享一张 GPU，一台节点包含多张 GPU，多台节点通过网络连接。算术在数据上执行，通信把数据送到拥有下一段计算或状态的设备。上一讲从算法讨论复制与分片，本讲把这些动作写成 `collective` 和点对点代码。

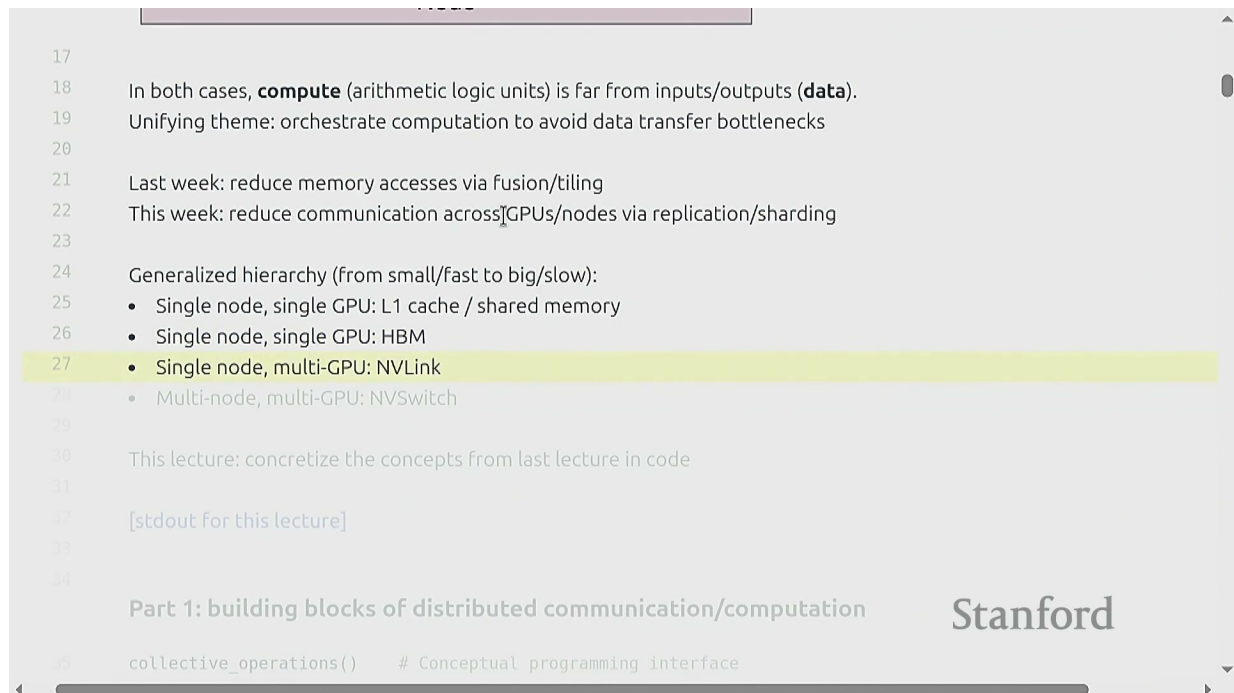


图 1: 单 GPU、单节点、多节点形成由快到慢的层级。

视频证据时间窗: 00:02:40-00:03:20。

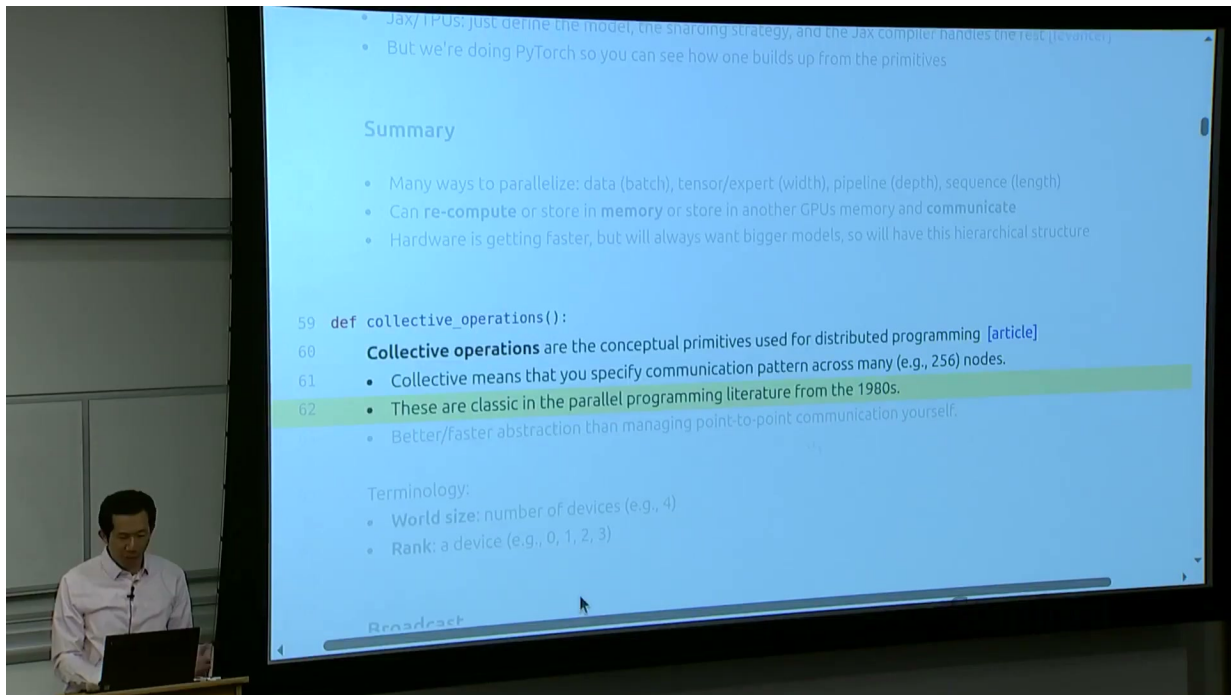


图 2: 从 collective、benchmark 到数据/张量/流水线并行的代码路线。

视频证据时间窗: 00:04:25–00:05:05。

设备层级可抽象为图 $G = (V, E)$, 一次映射的通信代价为

$$C(\pi) = \sum_{(u,v) \in E_{\text{alg}}} \frac{S_{uv}}{B_{\pi(u), \pi(v)}}.$$

- E_{alg} : 并行算法需要通信的逻辑边;
- S_{uv} : 边上张量字节数;
- π : 逻辑 rank 到物理设备的映射;
- $B_{\pi(u), \pi(v)}$: 对应物理链路有效带宽。

进程、rank 和 GPU 必须一一说清

`world_size` 是进程组大小, `rank` 是组内编号, `local_rank` 常用于选择本机 GPU。多进程共用一张卡或 rank 绑定错误, 都可能让程序“能运行但性能完全失真”。

若每节点 g 张 GPU、节点数为 n , 常见世界大小为

$$p = ng.$$

- p : 进程组中的 rank 总数;
- g : 每节点 GPU 数;
- n : 节点数;
- 还可建立子进程组分别承载 TP、PP、DP。

1.2 本章小结

并行代码的第一步不是调用 all-reduce，而是定义进程拓扑、设备绑定和通信组。算法的逻辑边必须映射到真实硬件链路。

2 集合通信的语义

2.1 一对多、多对一与全体交换

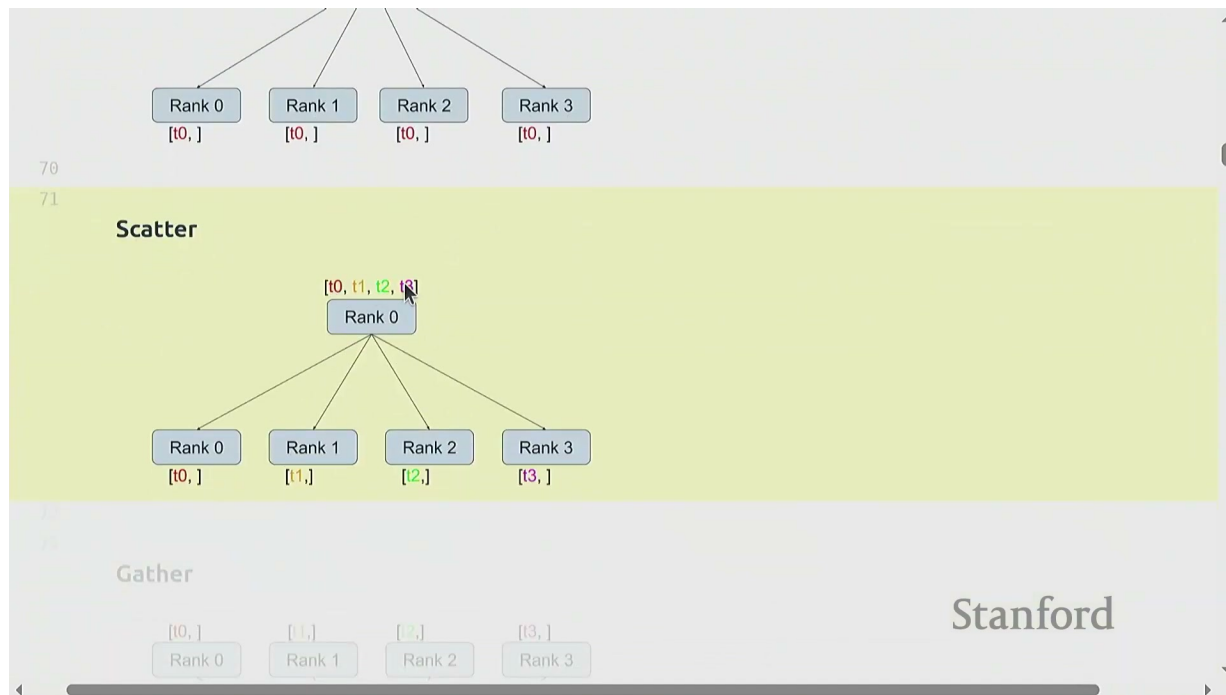


图 3: broadcast: 根 rank 的张量复制到所有 rank。

视频证据时间窗: 00:05:25–00:06:05。

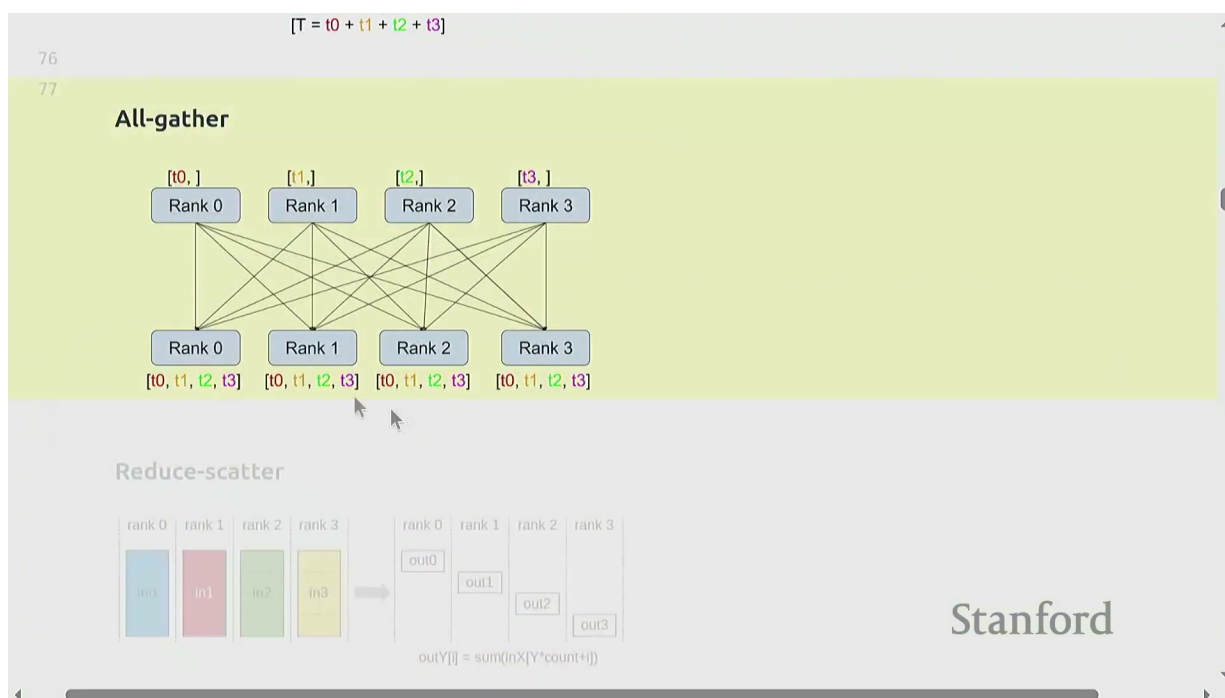


图 4: scatter: 根 rank 将不同分片发送给各 rank。

视频证据时间窗: 00:05:55–00:06:35。

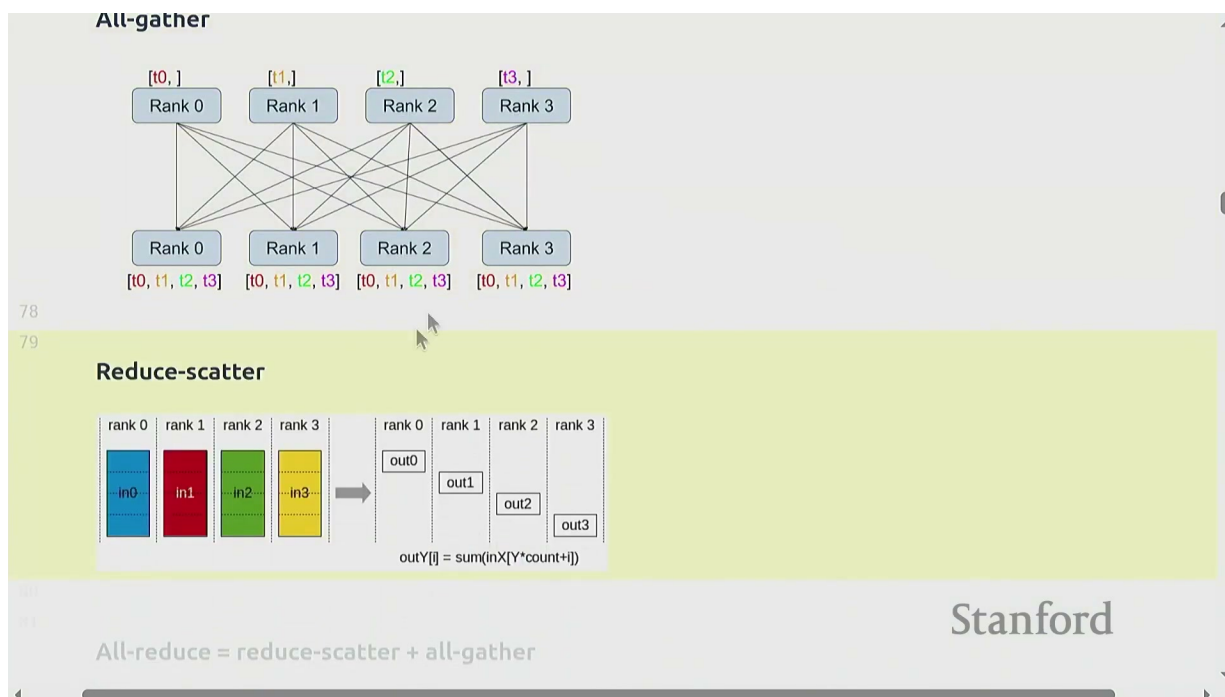


图 5: gather: 各 rank 的分片集中到根 rank。

视频证据时间窗: 00:06:10–00:06:50。

若根 rank 为 r_0 , broadcast 的结果是

$$y_r = x_{r_0}, \quad r = 0, \dots, p-1.$$

- x_{r_0} : 根进程输入;

- y_r : 每个进程的输出;
- 可用于初始化参数、配置或随机种子状态。

scatter/gather 对张量分片 $\{x_0, \dots, x_{p-1}\}$ 满足

$$\text{gather}(\text{scatter}(x)) = x.$$

- 恒等关系假设分片顺序与形状约定一致;
- scatter 的每个 rank 只得到一片;
- gather 只有根 rank 得到完整结果。

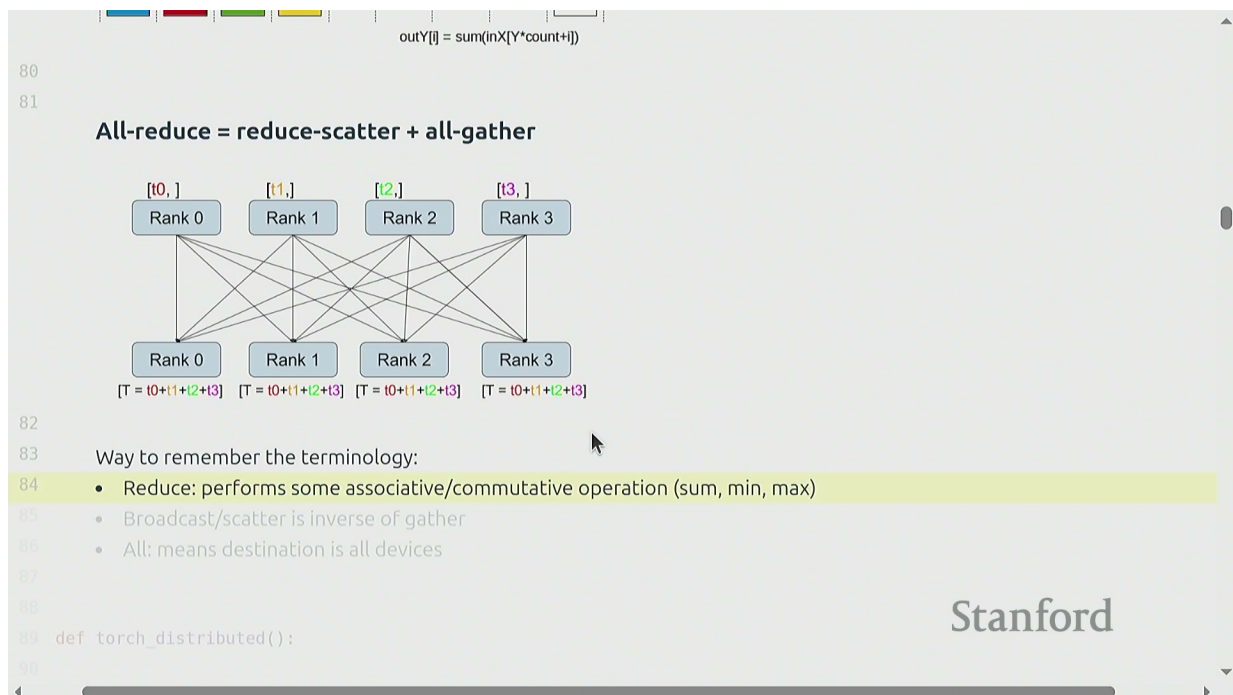


图 6: all-gather: 每个 rank 最终得到所有分片。

视频证据时间窗: 00:07:10–00:07:50。

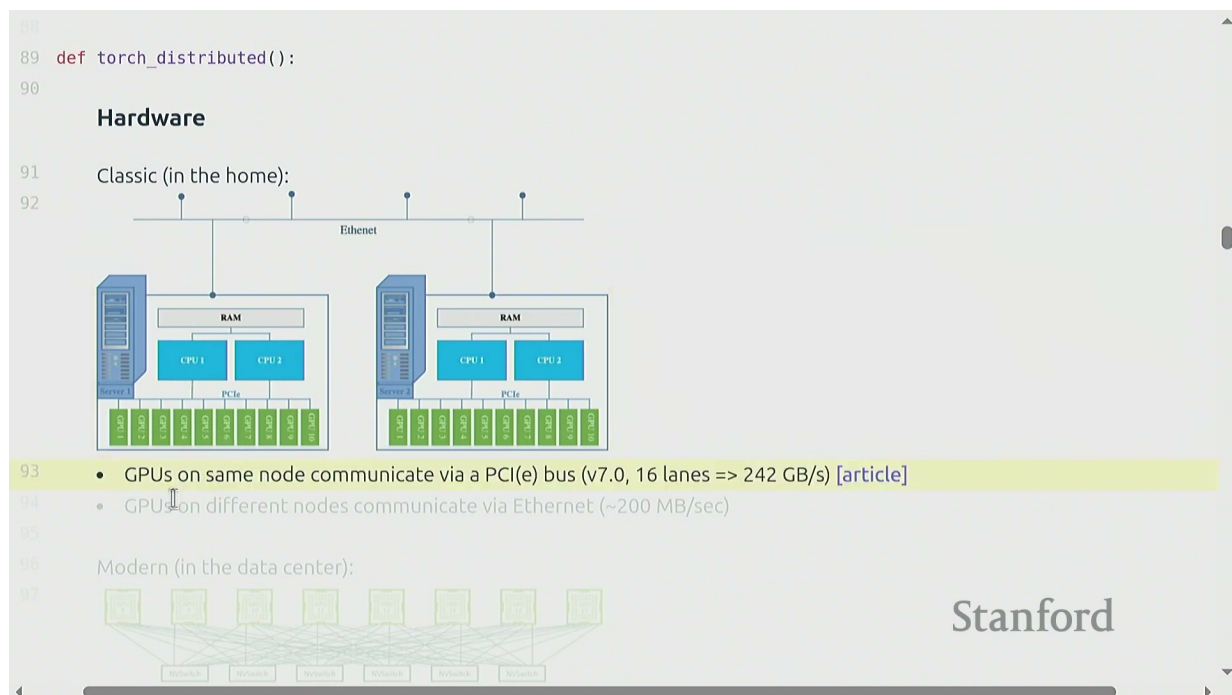


图 7: reduce-scatter: 先归约对应块, 再将结果块分散。

视频证据时间窗: 00:08:10–00:08:50。

all-gather 的每 rank 输出大小为

$$S_{\text{out}} = p S_{\text{shard}}.$$

- S_{shard} : 单 rank 输入分片字节数;
- 输出沿约定维度拼接;
- FSDP 用它按需物化完整参数。

reduce-scatter 对第 k 个分片计算

$$y_k = \bigoplus_{r=0}^{p-1} x_{r,k}.$$

- $x_{r,k}$: rank r 的第 k 片输入;
- y_k : 只保留在 rank k 的归约结果;
- ZeRO 用它让每个 rank 只拥有一片梯度。

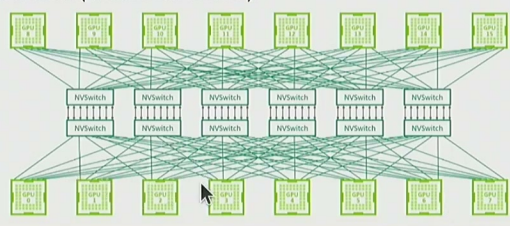
93 • GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes => 242 GB/s) [article]

94 • GPUs on different nodes communicate via Ethernet (~200 MB/sec)

95

96 Modern (in the data center):

97



98 • Within a node: NVLink connects GPUs directly, bypass CPU

99 • Across nodes: NVSwitch connects GPUs directly, bypass Ethernet

100

101 Each H100 has 18 NVLink 4.0 links, for a total of 900GB/s [article]

102 In comparison, memory bandwidth for HBM is 3.9 TB/s [article]

103

104 Let's check what our hardware setup is. [article]

105 `if torch.cuda.is_available():`

106 `os.system("nvidia-smi topo -m")`

Stanford

图 8: all-reduce 等价于 reduce-scatter 加 all-gather。

视频证据时间窗：00:09:25–00:10:05。

语义测试应检查每个 rank

分布式 bug 常只在某个 rank 出现。单元测试必须打印或断言每个 rank 的输入、输出、shape、dtype 和 device；只在 rank 0 检查会遗漏错序、未初始化和错误分片。

原语	谁得到完整结果	典型用途	常见错误
broadcast	所有 rank	状态复制	根 rank 不一致
scatter	无人	数据/状态分片	列表长度不等于 p
gather	仅根 rank	集中结果	非根误读输出
all-gather	所有 rank	参数物化	拼接维度错误
reduce-scatter	无人	梯度分片	分片形状不一致
all-reduce	所有 rank	DDP 梯度同步	忘记除以 p

2.2 本章小结

collective 的区别不是 API 名称，而是输入如何分片、在哪里归约、谁拥有完整输出。写并行算法时应先画每个 rank 的张量状态，再选择原语。

3 拓扑、NCCL 与 `torch.distributed`

3.1 同节点快，跨节点慢

93 • GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes => 242 GB/s) [article]

94 • GPUs on different nodes communicate via Ethernet (~200 MB/sec)

95

96 Modern (in the data center):

97

98 • Within a node: NVLink connects GPUs directly, bypass CPU

99 • Across nodes: NVSwitch connects GPUs directly, bypass Ethernet

100

101 Each H100 has 18 NVLink 4.0 links, for a total of 900GB/s [article]

102 In comparison, memory bandwidth for HBM is 3.9 TB/s [article]

103

104 Let's check what our hardware setup is. [article]

Stanford

图 9: 经典单机通过 PCIe 总线连接 CPU 与 GPU。

视频证据时间窗: 00:10:25–00:11:05。

90 Hardware

91 Classic (in the home):

92

93 • GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes => 242 GB/s) [article]

94 • GPUs on different nodes communicate via Ethernet (~200 MB/sec)

95

96 Modern (in the data center):

97

Stanford

图 10: 现代节点内使用 NVLink/NVSwitch, 节点间再经网络。

视频证据时间窗: 00:11:40–00:12:20。

一条消息大小为 S ，通信时间近似

$$T = \alpha + \frac{S}{B}.$$

- α : 启动和传播延迟;
- B : 有效带宽;
- 小消息由 α 主导, 大消息由 S/B 主导。

课程列出 H100 级节点内 NVLink 总带宽约 900 GB/s, 而 HBM 带宽约 3.9 TB/s。两者比值为

$$\rho = \frac{900}{3900} \approx 0.23.$$

- 900 GB/s: 课件给出的每 H100 NVLink 量级;
- 3.9 TB/s: 课件用于对照的显存带宽量级;
- 即使高速互连也显著慢于本地 HBM, 通信不能当成本地读取。

宣传带宽不是 **collective** 带宽

链路数字可能是双向总和、单向峰值或聚合带宽。**collective** 还受拓扑、协议、消息分块和并发影响。基准应报告算法带宽与总线带宽的口径, 不能直接拿时间除以原始张量大小就结束。

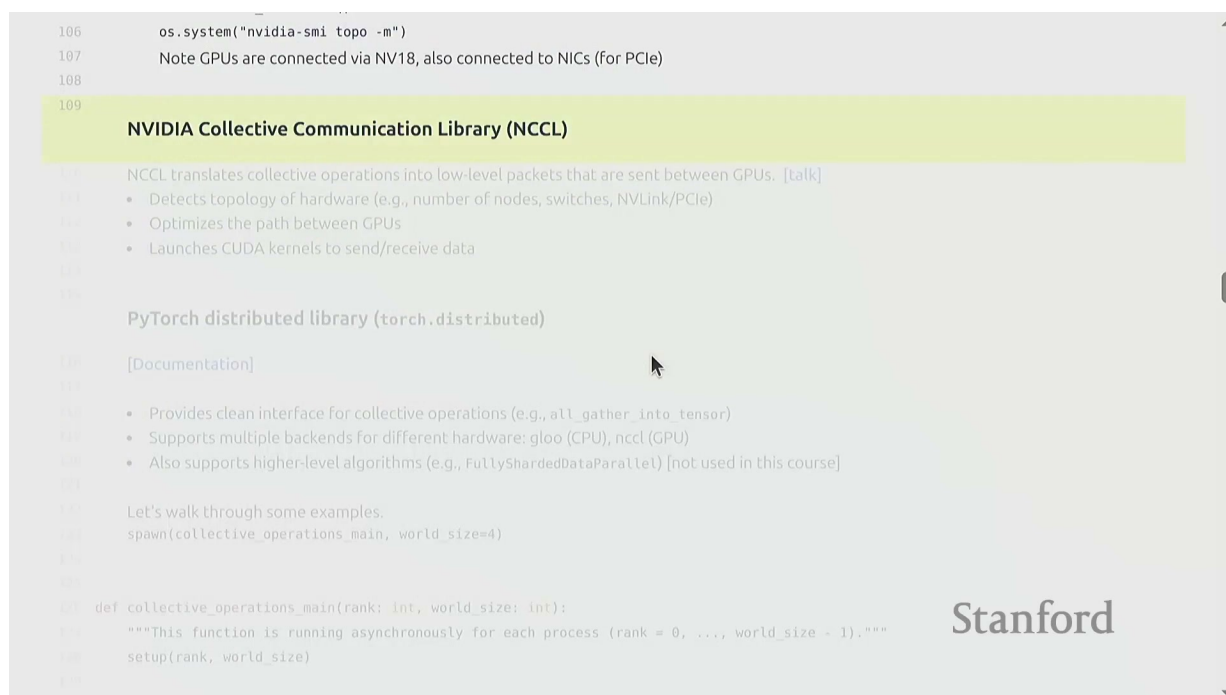
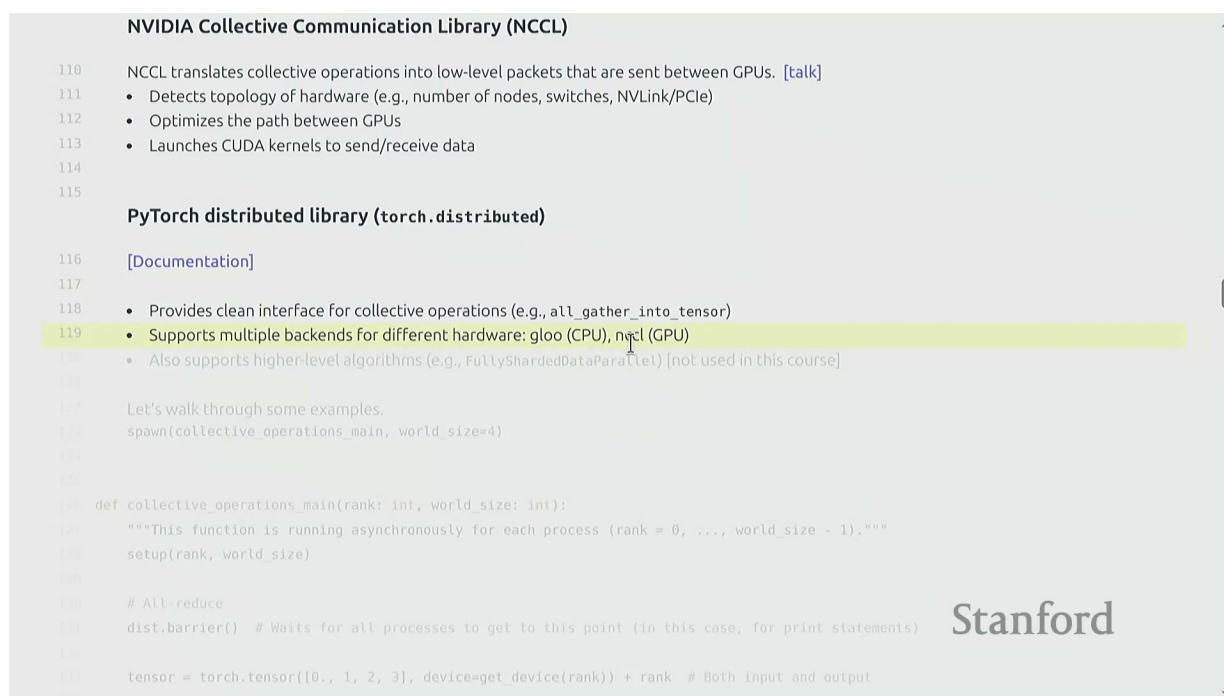


图 11: NCCL 根据拓扑为 GPU collective 选择底层算法。

视频证据时间窗: 00:13:15–00:13:55。

图 12: `torch.distributed` 提供进程组和 collective 的 Python 接口。

视频证据时间窗: 00:15:00–00:15:40。

Listing 1: 初始化进程组与绑定设备的最小结构

```

1 dist.init_process_group(backend="nccl")
2 rank = dist.get_rank()
3 world = dist.get_world_size()
4 local_rank = int(os.environ["LOCAL_RANK"])
5 torch.cuda.set_device(local_rank)
6 device = torch.device("cuda", local_rank)

```

NCCL 是执行层，不替代算法设计

NCCL 会拓扑感知地执行 collective，但不会替你决定张量该复制还是分片、通信何时发生、是否能与计算重叠。高性能训练仍需在上层设计 bucket、process group 和调度。

3.2 本章小结

`torch.distributed` 定义语义，NCCL 执行 GPU 通信，硬件拓扑给出速度边界。三层都要记录，性能结论才可迁移。

4 编写并验证 collective 实验

4.1 让每个 rank 的输入可辨认

```

122 Let's walk through some examples.
123 spawn(collective_operations_main, world_size=4)
124
125
126 def collective_operations_main(rank: int, world_size: int):
127     """This function is running asynchronously for each process (rank = 0, ..., world_size - 1)."""
128     setup(rank, world_size)
129
130     # All-reduce
131     dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)
132
133     tensor = torch.tensor([0., 1, 2, 3], device=get_device(rank)) + rank # Both input and output
134
135     print(f"Rank {rank} [before all-reduce]: {tensor}", flush=True)
136     dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place
137     print(f"Rank {rank} [after all-reduce]: {tensor}", flush=True)
138
139     # Reduce-scatter
140     dist.barrier()
141
142     input = torch.arange(world_size, dtype=torch.float32, device=get_device(rank)) + rank # Input
143     output = torch.empty(1, device=get_device(rank)) # Allocate output
144
145     print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)
146     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
147     print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)
148
149     # All-gather

```

图 13: 测试函数为各 rank 构造不同值，再逐项调用 collective。

视频证据时间窗：00:16:40–00:17:20。

```

126 def collective_operations_main(rank: int, world_size: int):
127     """This function is running asynchronously for each process (rank = 0, ..., world_size - 1)."""
128     setup(rank, world_size)
129
130     # All-reduce
131     dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)
132
133     tensor = torch.tensor([0., 1, 2, 3], device=get_device(rank)) + rank # Both input and output
134
135     print(f"Rank {rank} [before all-reduce]: {tensor}", flush=True)
136     dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place
137     print(f"Rank {rank} [after all-reduce]: {tensor}", flush=True)
138
139     # Reduce-scatter
140     dist.barrier()
141
142     input = torch.arange(world_size, dtype=torch.float32, device=get_device(rank)) + rank # Input
143     output = torch.empty(1, device=get_device(rank)) # Allocate output
144
145     print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)
146     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
147     print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)
148
149     # All-gather
150     dist.barrier()
151
152     input = output # Input is the output of reduce-scatter

```

图 14: all-reduce 前后打印张量，直接验证求和语义。

视频证据时间窗：00:18:15–00:18:55。

若 rank r 初始化标量 $x_r = r$, 求和 all-reduce 后应得到

$$y = \sum_{r=0}^{p-1} r = \frac{p(p-1)}{2}.$$

- p : world size;
- 每个 rank 都应得到相同的 y ;
- 这个可预测输入比随机张量更容易定位错误。

Listing 2: 可判定的 all-reduce 语义测试

```

1 x = torch.tensor([float(rank)], device=device)
2 dist.all_reduce(x, op=dist.ReduceOp.SUM)
3 expected = world_size * (world_size - 1) / 2
4 assert x.item() == expected

```

barrier 只用于建立实验边界

barrier 可让所有 rank 在同一点开始或结束, 便于读日志和测量; 在训练热路径滥用会把原本可异步的工作强制串行。若 **collective** 本身已经形成依赖, 不要再无条件添加全局屏障。

若 **collective** 对所有 rank 都正确, 集合不变量可写为

$$\forall r, \text{shape}(y_r) = \text{expected_shape}(r).$$

- 某些原语各 rank 输出相同, 某些输出只是不同分片;
- 形状、值、顺序、dtype、device 都应纳入断言;
- 超时与异常也应带 rank 标识输出。

日志输出会扰动时序

多 rank 同时打印会锁竞争、打乱顺序并显著拖慢小实验。语义调试阶段可按 rank 顺序 barrier 后打印; 性能阶段则关闭热路径日志, 只保留聚合统计。

4.2 本章小结

collective 实验必须可判定: 为每个 rank 构造不同输入, 用闭式结果验证输出, 并把 barrier 限制在测试边界。先证明语义, 才有资格测速度。

5 通信基准与带宽口径

5.1 异步 CUDA 与同步测量

```

131     dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)
132
133     tensor = torch.tensor([0., 1, 2, 3], device=get_device(rank)) + rank # Both input and output
134
135     print(f"Rank {rank} [before all-reduce]: {tensor}", flush=True)
136     dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place
137     print(f"Rank {rank} [after all-reduce]: {tensor}", flush=True)
138
139     # Reduce-scatter
140     dist.barrier()
141
142     input = torch.arange(world_size, dtype=torch.float32, device=get_device(rank)) + rank # Input
143     output = torch.empty(1, device=get_device(rank)) # Allocate output
144
145     print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)
146     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
147     print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)
148
149     # All-gather
150     dist.barrier()
151
152     input = output # Input is the output of reduce-scatter
153     output = torch.empty(world_size, device=get_device(rank)) # Allocate output
154
155     print(f"Rank {rank} [before all-gather]: input = {input}, output = {output}", flush=True)
156     dist.all_gather_into_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
157     print(f"Rank {rank} [after all-gather]: input = {input}, output = {output}", flush=True)
158

```

Stanford

图 15: 计时前后同步 CUDA, 并让所有 rank 对齐实验阶段。

视频证据时间窗: 00:20:10-00:20:50。

重复 n 次的平均时间为

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i.$$

- t_i : 预热后一次 collective 的同步时间;
- 所有 rank 应报告最大或分位数, 不能只取最快 rank;
- 慢 rank 决定同步训练的关键路径。

若张量有 N 个元素、每元素 s 字节, payload 为

$$S = Ns.$$

- N : `tensor.numel()`;
- s : `tensor.element_size()`;
- 这是用户数据大小, 不等于网络总线实际传输量。


```

131 dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)
132
133 tensor = torch.tensor([0., 1, 2, 3], device=get_device(rank)) + rank # Both input and output
134
135 print(f"Rank {rank} [before all-reduce]: {tensor}", flush=True)
136 dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place
137 print(f"Rank {rank} [after all-reduce]: {tensor}", flush=True)
138
139 # Reduce-scatter
140 dist.barrier()
141
142 input = torch.arange(world_size, dtype=torch.float32, device=get_device(rank)) + rank # Input
143 output = torch.empty(1, device=get_device(rank)) # Allocate output
144
145 print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)
146 dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
147 print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)
148
149 # All-gather
150 dist.barrier()
151
152 input = output # Input is the output of reduce-scatter
153 output = torch.empty(world_size, device=get_device(rank)) # Allocate output
154
155 print(f"Rank {rank} [before all-gather]: input = {input}, output = {output}", flush=True)
156 dist.all_gather_into_tensor(output_tensor=output, input_tensor=input, async_op=False)
157 print(f"Rank {rank} [after all-gather]: input = {input}, output = {output}", flush=True)
158

```

Stanford

图 16: 代码从元素数和元素字节数计算消息 payload。

视频证据时间窗: 00:22:10-00:22:50。

对 ring all-reduce, 算法带宽可定义为

$$B_{\text{alg}} = \frac{S}{t},$$

- S : 输入张量大小;
- t : collective 时间;
- 该口径便于从用户张量理解吞吐。

折算总线带宽为

$$B_{\text{bus}} = 2 \frac{p-1}{p} B_{\text{alg}}.$$

- 系数反映 reduce-scatter 与 all-gather 的链路流量;
- 用它更适合与硬件链路峰值比较;
- 必须在报告中注明采用的 collective 算法假设。

```

192 dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False)
193 if torch.cuda.is_available():
194     torch.cuda.synchronize() # Wait for CUDA kernels to finish
195     dist.barrier()           # Wait for all the processes to get here
196 end_time = time.time()
197
198 duration = end_time - start_time
199 print(f"[all_reduce] Rank {rank}: all_reduce(world_size={world_size}, num_elements={num_elements}) took
{render_duration(duration)}", flush=True)
200
201 # Measure the effective bandwidth
202 dist.barrier()
203 size_bytes = tensor.element_size() * tensor.numel()
204 sent_bytes = size_bytes * 2 * (world_size - 1) # 2x because send input and receive output
205 total_duration = world_size * duration
206 bandwidth = sent_bytes / total_duration
207 print(f"[all_reduce] Rank {rank}: all_reduce measured bandwidth = {round(bandwidth / 1024**3)} GB/sec", flush=True)
208
209 cleanup()
210
211 def reduce_scatter(rank: int, world_size: int, num_elements: int):
212     setup(rank, world_size)
213
214     # Create input and outputs
215     input = torch.randn(world_size, num_elements, device=get_device(rank)) # Each rank has a matrix
216     output = torch.empty(num_elements, device=get_device(rank))

```

图 17: 实验按张量尺寸扫描 all-reduce 时间和带宽。

视频证据时间窗: 00:27:25–00:28:05。

性能统计取最慢 rank

同步 collective 完成时间由最慢参与者决定。每个 rank 独立计时后，应 all-reduce MAX 得到全局步时；平均 rank 时间会掩盖网络抖动、NUMA 错绑或单卡降频。

速度提升相对理论上限可写为

$$\eta_B = \frac{B_{\text{bus}}}{B_{\text{link}}}.$$

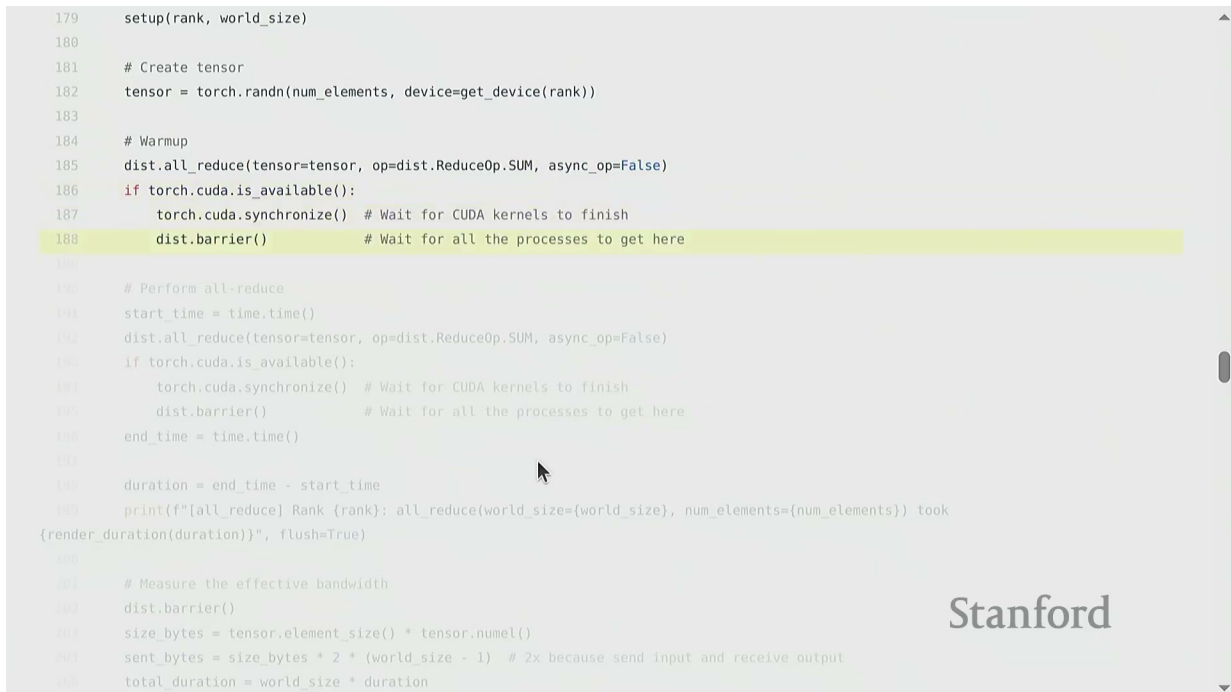
- B_{link} : 同口径下物理链路可持续带宽;
- η_B : 通信利用率;
- 小消息下延迟占主导，该比值自然很低。

5.2 本章小结

通信 benchmark 的关键是同步边界、payload 计算、最慢 rank 和带宽口径。单个“GB/s”若没有这四项说明，无法与拓扑或其他系统比较。

6 跨节点通信与分组

6.1 高频原语应留在高速域



```

179     setup(rank, world_size)
180
181     # Create tensor
182     tensor = torch.randn(num_elements, device=get_device(rank))
183
184     # Warmup
185     dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False)
186     if torch.cuda.is_available():
187         torch.cuda.synchronize() # Wait for CUDA kernels to finish
188         dist.barrier()           # Wait for all the processes to get here
189
190     # Perform all-reduce
191     start_time = time.time()
192     dist.all_reduce(tensor=tensor, op=dist.ReduceOp.SUM, async_op=False)
193     if torch.cuda.is_available():
194         torch.cuda.synchronize() # Wait for CUDA kernels to finish
195         dist.barrier()           # Wait for all the processes to get here
196     end_time = time.time()
197
198     duration = end_time - start_time
199     print(f"[all_reduce] Rank {rank}: all_reduce(world_size={world_size}, num_elements={num_elements}) took
{render_duration(duration)}", flush=True)
200
201     # Measure the effective bandwidth
202     dist.barrier()
203     size_bytes = tensor.element_size() * tensor.numel()
204     sent_bytes = size_bytes * 2 * (world_size - 1) # 2x because send input and receive output
205     total_duration = world_size * duration

```

图 18: 跨节点实验暴露节点内与节点间带宽差异。

视频证据时间窗: 00:24:25–00:25:05。

若一个并行步骤包含节点内通信 V_{intra} 和节点间通信 V_{inter} , 时间近似

$$T_{\text{comm}} \approx \frac{V_{\text{intra}}}{B_{\text{intra}}} + \frac{V_{\text{inter}}}{B_{\text{inter}}}.$$

- B_{intra} : NVLink/NVSwitch 等节点内有效带宽;
- B_{inter} : InfiniBand/Ethernet 等节点间有效带宽;
- 通常 $B_{\text{intra}} \gg B_{\text{inter}}$, 因此少量跨节点字节也可主导。

将 p 个 rank 划成大小为 q 的局部组, 组数为

$$n_{\text{group}} = \frac{p}{q}.$$

- q : 例如每节点 GPU 数或张量并行度;
- 每组创建独立 process group;
- 不同并行轴在不同组内调用 collective。

层级 collective

大规模 all-reduce 可以先节点内 reduce, 再跨节点 reduce, 最后节点内 broadcast; 也可使用分层 reduce-scatter/all-gather。核心是减少慢链路上的参与者或字节, 同时充分利用节点内高速互连。

process group 次序必须全局一致

所有 rank 创建通信组的顺序若不一致，可能永久等待。生产代码应由统一拓扑描述生成所有组，记录每个组的成员，并在启动阶段做一致性断言。

6.2 本章小结

跨节点扩展的本质是拓扑映射。把高频、大张量同步限制在节点内，把低频边界通信放到节点间，并通过层级 collective 降低慢链路压力。

7 手写数据并行与状态所有权**7.1 复制参数、切数据、归约梯度**

```

219 # Warmup
220 dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
221 if torch.cuda.is_available():
222     torch.cuda.synchronize() # Wait for CUDA kernels to finish
223     dist.barrier()           # Wait for all the processes to get here
224
225 # Perform reduce-scatter
226 start_time = time.time()
227 dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
228 if torch.cuda.is_available():
229     torch.cuda.synchronize() # Wait for CUDA kernels to finish
230     dist.barrier()           # Wait for all the processes to get here
231 end_time = time.time()
232
233 duration = end_time - start_time
234 print(f"[reduce_scatter] Rank {rank}: reduce_scatter(world_size={world_size}, num_elements={num_elements}) took
{render_duration(duration)}", flush=True)
235
236 # Measure the effective bandwidth
237 dist.barrier()
238 data_bytes = output.element_size() * output.numel() # How much data in the output
239 sent_bytes = data_bytes * (world_size - 1) # How much needs to be sent (no 2x here)
240 total_duration = world_size * duration # Total time for transmission
241 bandwidth = sent_bytes / total_duration
242 print(f"[reduce_scatter] Rank {rank}: reduce_scatter measured bandwidth = {round(bandwidth / 1024**3)} GB/sec", flush=True)
243
244 cleanup()

```

图 19: 课程用简化 MLP 逐步实现数据、张量和流水线并行。

视频证据时间窗：00:31:25–00:32:05。

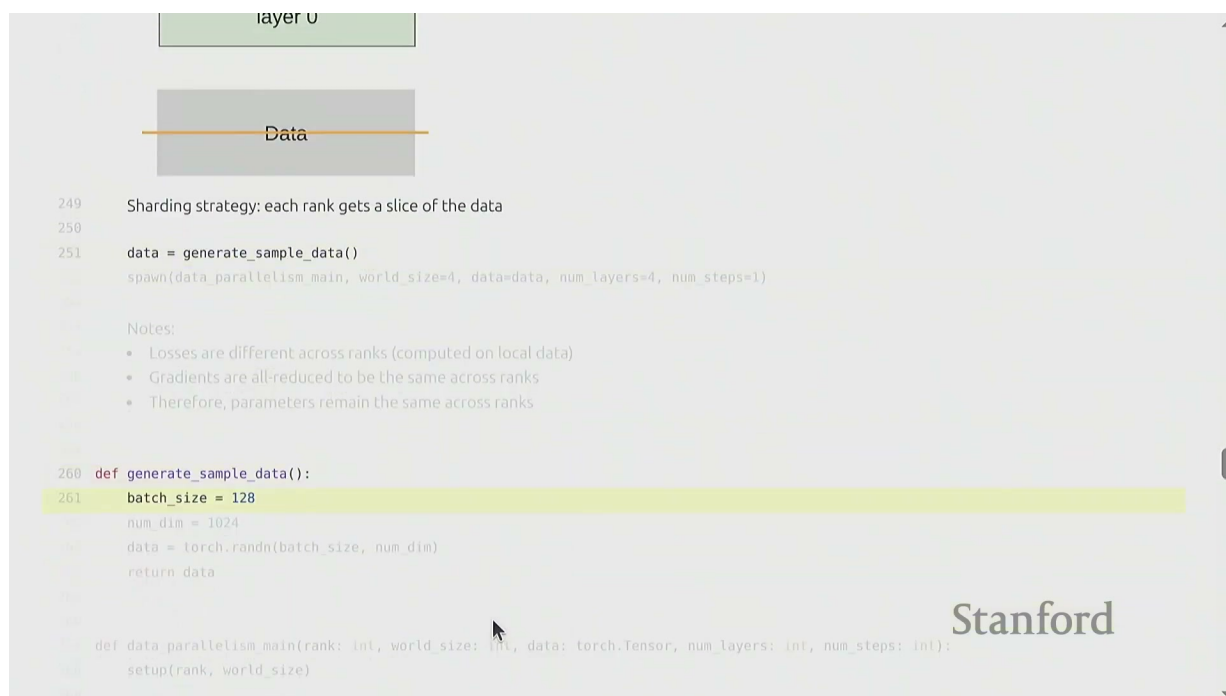


图 20: 数据并行切 batch, 并在更新前同步梯度。

视频证据时间窗: 00:34:00-00:34:40。

若全局 batch 为 B 、world size 为 p , 均匀局部 batch 为

$$B_{\text{local}} = \frac{B}{p}.$$

- 需保证 B 可整除或定义余数策略;
- 每个 rank 使用不同数据切片;
- 参数初始化必须相同。

局部梯度 g_r all-reduce 求和后, 若损失在各 rank 内已取均值, 则全局平均梯度为

$$g = \frac{1}{p} \sum_{r=0}^{p-1} g_r.$$

- 是否除以 p 取决于本地损失和 collective 的约定;
- 重复缩放或漏缩放都会改变有效学习率;
- 应与单卡同全局 batch 的一步更新做数值对照。

Listing 3: 手写同步数据并行的核心

```

1 loss.backward()
2 for p in model.parameters():
3     dist.all_reduce(p.grad, op=dist.ReduceOp.SUM)
4     p.grad.div_(world_size)
5 optimizer.step()
  
```

正确性金标准：单卡等价一步

固定随机种子和全局 batch，把多卡数据切片后执行一步，再与单卡相同 batch 的参数更新比较。该测试可同时发现样本重复、梯度缩放、初始化、dropout RNG 和优化器状态错误。

参数级通信量近似

$$V_{\text{DDP}} \approx 2 \frac{p-1}{p} s_g P.$$

- P : 参数个数;
- s_g : 梯度字节;
- 与 batch 大小无关，所以局部 batch 过小时通信占比上升。

真实 DDP 还做了什么

框架会把梯度分 bucket，在反向过程中逐桶 all-reduce，以通信覆盖计算；还处理 unused parameters、梯度视图和异常。手写循环适理解语义，不应被误认为生产 DDP 的完整替代。

7.2 本章小结

数据并行的代码核心很短，真正困难的是保证数据互斥、参数一致、梯度缩放正确，并把参数级通信挂到反向时间线上。

8 张量并行与流水线并行的代码形状

8.1 张量并行：切权重宽度

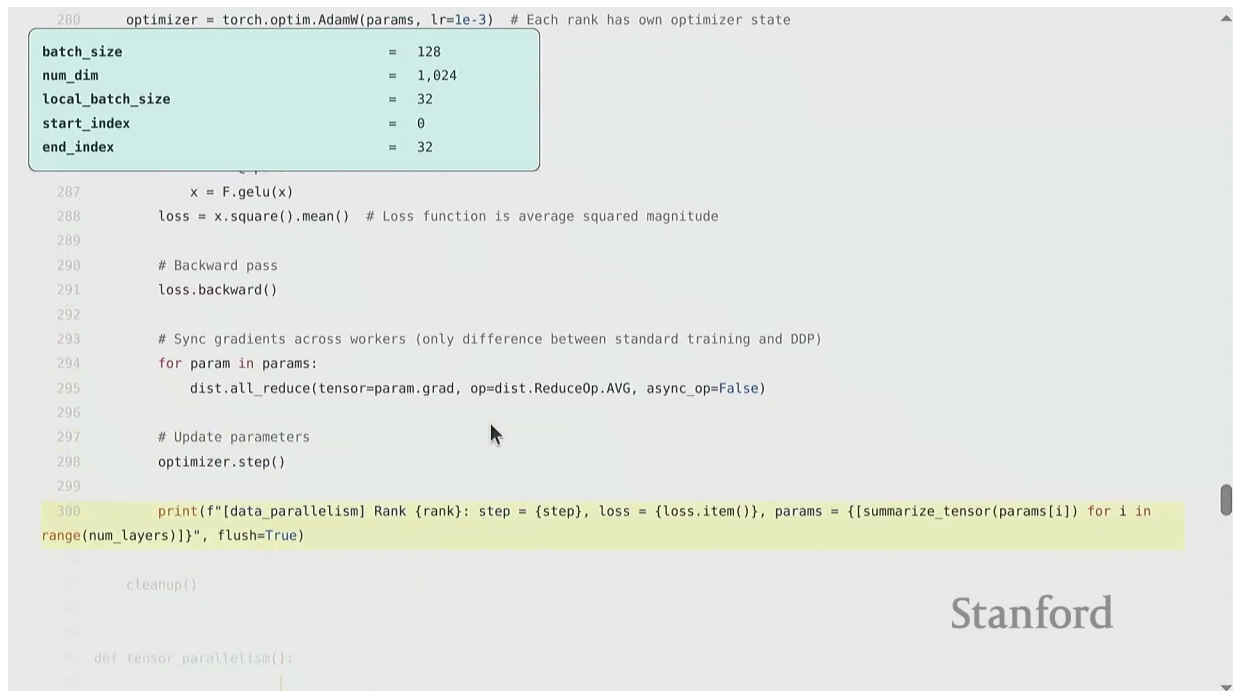


图 21: 张量并行让不同 rank 保存线性层的不同列或行。

视频证据时间窗：00:37:45–00:38:25。

列并行线性层把 $W = [W_0, \dots, W_{p-1}]$ 分片：

$$y_r = xW_r.$$

- 每个 rank 保留 $1/p$ 输出列；
- 若下一层可消费分片输出，无需立即 all-gather；
- 否则通过 all-gather 拼成完整 y 。

行并行将输入和权重按收缩维切分：

$$y = \sum_{r=0}^{p-1} x_r W_r.$$

- 每个 rank 产生部分和；
- all-reduce 汇总最终输出；
- MLP 常把列并行和行并行配对，减少中间聚合。

Listing 4: 列并行线性层的核心轮廓

```

1 local_w = shard_columns(full_w, rank, world_size)
2 local_y = x @ local_w
3 parts = [torch.empty_like(local_y) for _ in range(world_size)]
4 dist.all_gather(parts, local_y)
5 y = torch.cat(parts, dim=-1)

```

Center for Research on Foundation Models | HAI | Stanford University Human-Centered Artificial Intelligence

People Report Research Policy Blog

- `param_mapping`, a `dict` that specifies how we shard our parameters and optimizer states. We shard along the `embed` axis, which is the hidden state of our transformer.
- `data_mapping`, a `dict` that specifies how we shard both our inputs and intermediate states during computation.

Tensor Parallelism in the Same 10 Lines of Code

Let's further add tensor parallelism to our model. All we have to do is change the axis mappings:

```

 1  + # Specify which axes we shard for tensor parallelism:
 2  + # specifying "head" shards attention and "mlp" shards the feedforward
 3  + tensor_parallel_mapping = {"head": "model", "mlp": "model"}
 4  # We store our parameters and optimizer states fully sharded along the embed axis
 5  - param_mapping = {"embed": "data"}
 6  + param_mapping = {"embed": "data", **tensor_parallel_mapping}
 7  # During computation, we instead shard our data along the batch axis, and gather the parameters just-in-time
 8  - data_mapping = {"batch": "data"}
 9  + data_mapping = {"batch": "data", **tensor_parallel_mapping}

```

That's it! We can now use a combination of tensor parallelism and FSDP to scale our model to as many GPUs or TPUs as we want. By comparison, in PyTorch, there are usually significant changes to the model required to add tensor parallelism, including replacing the right set of `Linear` layers with modified versions that do communication.

图 22: 课件展示用少量代码在权重上定义张量并行映射。

视频证据时间窗: 01:00:05–01:00:45。

切分维度必须贯穿前后向

只让前向 `shape` 对上还不够。偏置属于哪一片、激活函数在哪里执行、反向梯度如何聚合、优化器只更新哪一片，都必须与布局一致。最好为每层写单卡等价测试和分片形状断言。

8.2 流水线并行：切模型深度

```

314     setup(rank, world_size)

    batch_size      = 128
    num_dim         = 1,024
    local_num_dim   = 256

319     local_num_dim = int_divide(num_dim, world_size) # Shard 'num_dim' @inspect local_num_dim
320
321     # Create model (each rank gets 1/world_size of the parameters)
322     params = [get_init_params(num_dim, local_num_dim, rank) for i in range(num_layers)]
323
324     # Forward pass
325     x = data
326     for i in range(num_layers):
327         # Compute activations (batch_size x local_num_dim)
328         x = x @ params[i] # Note: this is only on a slice of the parameters
329         x = F.gelu(x)
330
331         # Allocate memory for activations (world_size x batch_size x local_num_dim)
332         activations = [torch.empty(batch_size, local_num_dim, device=get_device(rank)) for _ in range(world_size)]
333
334         # Send activations via all gather
335         dist.all_gather(tensor_list=activations, tensor=x, async_op=False)
336
337         # Concatenate them to get batch_size x num_dim
338         x = torch.cat(activations, dim=1)
339
340     print(f"[tensor_parallelism] Rank {rank}: forward pass produced activations {summarize_tensor(x)}", flush=True)

```

图 23: 不同 rank 保存连续层段，并在边界发送激活。

视频证据时间窗：00:43:25–00:44:05。

若模型有 L 层，均匀切给 p 个 stage，理想每 stage 层数为

$$L_r \approx \frac{L}{p}.$$

- 实际应按执行时间和显存而非层数平分；
- 首尾 stage 还可能包含 embedding 和输出头；
- stage 边界张量必须显式发送与接收。

对于 m 个 microbatch 和 p 个 stage，简单流水线利用率近似

$$u_{\text{pipe}} = \frac{m}{m + p - 1}.$$

- m : microbatch 数；
- $p - 1$: 填充/排空气泡；
- 增大 m 可提高利用率，但会增加调度和激活管理压力。

```

374 # The data
batch_size      = 128
num_dim         = 1,024
local_num_layers = 2
micro_batch_size = 32

380 for x in micro_batches:
381     # Get activations from previous rank
382     if rank - 1 >= 0:
383         dist.recv(tensor=x, src=rank - 1)
384
385     # Compute layers assigned to this rank
386     for param in local_params:
387         x = x @ param
388         x = F.gelu(x)
389
390     # Send to the next rank (asynchronously)
391     if rank + 1 < world_size:
392         print(f"[pipeline_parallelism] Rank {rank}: sending {summarize_tensor(x)} to rank {rank + 1}", flush=True)
393         dist.send(tensor=x, dst=rank + 1)
394
395     Not handled: overlapping communication/computation to eliminate pipeline bubbles
396
397     # Backward pass: homework exercise
398
399 cleanup()

```

图 24: 代码按 microbatch 顺序 send/recv, 展示最基本的流水线调度。

视频证据时间窗: 00:50:10-00:50:50。

Listing 5: 流水线 stage 的概念伪代码

```

1 for micro in microbatches:
2     if rank > 0:
3         dist.recv(x, src=rank - 1)
4     y = local_layers(x)
5     if rank < world_size - 1:
6         dist.send(y, dst=rank + 1)

```

避免死锁要统一通信顺序

阻塞 send/recv 若两端顺序不匹配会互相等待。更稳健的实现使用显式 schedule、非阻塞 isend/irecv、固定 tag, 并在所有 stage 上用同一状态机推进 microbatch。

8.3 本章小结

张量并行改变层内张量布局, 流水线并行改变层的所有权和激活流。两者代码看似只是切片与 send/recv, 实则要求前后向、优化器、shape 和调度全链条一致。

9 把实验代码提升为训练系统

9.1 并行、显存与通信要共同回归

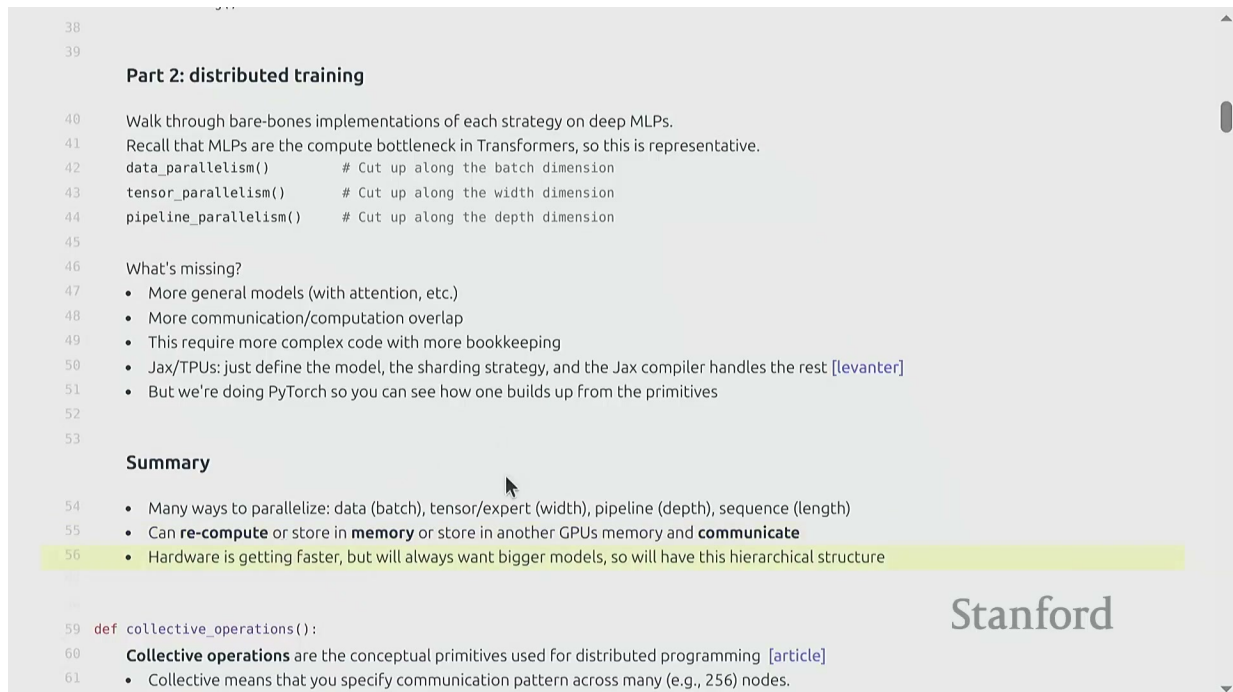


图 25: 课程总结: 多种切分方法、重计算和硬件层级必须一起考虑。

视频证据时间窗: 01:02:10–01:02:50。

组合并行的设备数满足

$$p = p_{DP} p_{TP} p_{PP}.$$

- p_{DP} : 数据并行组大小;
- p_{TP} : 张量并行组大小;
- p_{PP} : 流水线 stage 数;
- 每个 rank 同时属于多个互相正交的进程组。

每卡峰值显存必须满足

$$M_{\text{params}} + M_{\text{grads}} + M_{\text{optim}} + M_{\text{act}} + M_{\text{buffers}} \leq M_{\text{HBM}}.$$

- 前三项由复制/分片策略决定;
- 激活由 batch、序列长度、检查点和并行布局决定;
- buffer 包括 collective 临时区、workspace 和 allocator 碎片。

端到端 step 时间可分解为

$$T_{\text{step}} = T_{\text{compute}} + T_{\text{exposed comm}} + T_{\text{bubble}} + T_{\text{input}}.$$

- $T_{\text{exposed comm}}$: 无法与计算重叠的通信;

- T_{bubble} : 流水线、负载不均和同步空闲;
- T_{input} : 数据准备与主机提交;
- profiler 应能够为每一项提供时间线证据。

最小生产化回归集

一张卡与多卡一步参数等价; 不同 rank 的数据不重复; 每个分片 shape 和所有权正确; 故障时所有 rank 能退出; 典型形状的显存峰值不回退; collective 带宽和端到端 step time 有阈值。

复制课堂代码时要保留边界

本讲代码故意省略完整 Transformer、通信计算重叠、复杂 book-keeping、流水线反向调度和生产容错。它展示原理, 不是可直接提交到千卡集群的训练框架。

调优顺序

先证明单卡语义; 再证明多卡数值等价; 然后测 collective 和拓扑; 接着看显存峰值; 最后用时间线缩小暴露通信和空泡。若先追求吞吐, 错误的梯度缩放也可能给出漂亮数字。

9.2 本章小结

从 collective 到完整训练系统, 最难的不是 API, 而是维护跨 rank 的不变量: 谁拥有状态、何时物化、何时同步、如何缩放、怎样映射拓扑。把这些不变量写成测试, 再用带宽口径和时间线验证性能, 才能安全组合数据、张量和流水线并行。

10 分布式程序的调试与验收

10.1 启动阶段先证明身份

每个进程启动后应记录全局 rank、local rank、world size、主机名、进程号、CUDA device、GPU UUID 和所属进程组。随后执行一个极小 all-reduce, 把 rank 求和并与闭式结果比较。这个“冒烟 collective”能尽早发现端口、网络接口、设备绑定和组成员错误, 避免训练数分钟后才在第一轮梯度同步处挂起。

随机种子也有两类: 参数初始化常要求各 rank 相同, 数据采样和 dropout 又可能要求按 rank 或并行组可控地不同。应从一个全局种子派生模型种子、数据种子和算子种子, 并把派生规则写进检查点。否则重启或改变 world size 后, 即使参数成功恢复, 数据顺序和随机路径也无法复现。

所有异常都要带 rank

错误日志、shape 断言和超时信息必须包含 rank、组名和当前 collective 序号。分布式故障往往在一个进程首先发生, 其他进程只表现为等待; 没有首发 rank 的上下文, 最终看到的超时几乎没有诊断价值。

10.2 死锁的系统化排查

`collective` 要求同一进程组的所有 `rank` 以兼容顺序进入调用。若某个条件分支只在部分 `rank` 执行 `all-reduce`，或不同 `rank` 创建 `process group` 的顺序不同，就可能永久等待。点对点通信则要检查 `send/recv` 配对、源和目标、`tag`、张量 `shape` 与阻塞顺序。最有效的排查方法是给通信调用编号，在每个 `rank` 进入和返回时记录轻量事件。

不要用随处添加 `barrier` 的方式“修复”死锁。`barrier` 可能改变时序、掩盖真正不一致，并让性能更差。先找出第一个调用序号分叉的位置，再检查控制流和组成员。需要调试时可设置合理超时并启用通信库诊断日志，但生产运行应控制日志量，避免 I/O 反过来扰动通信。

shape 不同可能表现为挂起

某些后端不会立刻给出友好的维度错误；不同 `rank` 传入不同元素数、`dtype` 或 `collective` 类型，可能只表现为通信停滞。进入调用前的本地断言和一次元数据 `all-gather`，通常比等待超时更便宜。

10.3 数值等价的分层测试

第一层测试单个 `collective` 的输入输出；第二层测试一层并行线性或归一化与单卡参考相同；第三层测试一个前后向的激活和梯度；第四层测试一个优化步后的参数；最后才跑多步 `loss` 曲线。每层都固定数据和 `RNG`，使错误边界逐渐缩小。直接比较数百步训练 `loss`，差异出现时很难判断是通信、缩放、数据还是随机性。

数据并行重点检查每个样本恰好出现一次、梯度平均口径和优化器状态复制；张量并行检查切分维、偏置、归约位置和反向；流水线检查 `microbatch` 标识、激活与梯度配对、更新边界；`FSDP` 则检查参数物化、释放与检查点重分片。不同策略共享“单卡等价”原则，但不变量不同。

容差需要随归约变化

浮点加法不满足严格结合律，多卡归约顺序不同会产生微小差异。`FP32`、`BF16` 和 `FP16` 应设置不同容差，并同时看绝对误差、相对误差和长期 `loss`。容差不能大到掩盖漏除 `world size` 等系统性错误。

10.4 通信性能的对照实验

对每个物理层级分别测点对点、`all-reduce`、`all-gather` 和 `reduce-scatter`，扫描从小消息到大消息。小消息区估计延迟，大消息区估计持续带宽，拐点揭示协议切换。将 `rank` 绑定限制在单节点、同机架和跨节点三种范围，可直接量化拓扑差异。

训练中的通信通常与计算并发，独立 `benchmark` 只是上限。还要在真实模型时间线中检查 `collective` 是否进入独立 `CUDA stream`、是否被参数依赖正确等待、预取是否过早占用显存、最后一个 `bucket` 是否暴露。一个独立带宽很高的网络，若调度在计算结束后才启动，`step time` 仍会被完整通信相加。

保存两种带宽口径

算法带宽回答“用户张量多快完成”，总线带宽回答“链路利用得怎样”。报告同时保存消息大小、world size、原语、算法带宽、折算总线带宽和最慢 rank 时间，才能正确对比不同 collective。

10.5 检查点与弹性

分片训练的检查点不能只在 rank 0 保存当前可见张量。应决定保存完整状态还是分片状态，记录分片布局、全局 shape、dtype、优化器映射、数据加载位置和 RNG。完整状态易迁移但聚合成本高；分片状态写入并行，却需要明确的元数据与重分片工具。

恢复测试必须真的执行：在同 world size 恢复并比较下一步结果，再尝试目标支持的不同 world size；模拟一个 rank 中断，确认其他进程在有限时间内退出；检查临时文件不会被误认为完整 checkpoint。没有恢复演练的 checkpoint 只是文件，不是容错能力。

同步保存会制造长尾

若所有 rank 在 checkpoint 前后全局 barrier，最慢存储请求会阻塞整个作业。可使用分片异步写入和两阶段提交，但必须在 manifest 中区分“正在写”和“已完整”，防止恢复到不一致快照。

10.6 从课堂原型到框架

课堂手写循环故意显示数据切分、参数切分和 send/recv 的本质。生产框架还需要 bucket 化通信、计算通信重叠、混合精度、梯度累积、动态 loss scaling、激活检查点、编译图、故障传播和可观测性。理解原型的价值，是能在框架行为异常时画出每个 rank 的状态，而不是重复实现全部基础设施。

选择框架 API 时保留最小可观察接口：输出进程组、分片布局、显存峰值、collective 统计和 step 时间线；让测试能取到本地参数片和全局参考；让配置能表达物理拓扑。抽象若隐藏所有状态，调试会退化为试开关；抽象若暴露不变量，则能兼顾易用与证据。

最终验收清单

启动身份一致；collective 语义通过；单卡/多卡一步等价；真实形状无 OOM；最慢 rank 无异常长尾；通信组映射到预期拓扑；step time 分解可解释；检查点可恢复；异常能全局退出。完成这些项目，才算“分布式程序能工作”。

10.7 本章小结

分布式训练的可靠性来自显式不变量和分层验收。先验证身份与语义，再验证数值和性能，最后验证恢复与故障传播；每一步都保留 rank、组、shape、时间和拓扑证据。这样课堂中的短代码才能安全地成长为可运行、可解释、可恢复的训练系统。